1. 注册Kaggle并参加"Optiver✦ Realized Volatility Prediction"竞赛
2. 克隆qlib源码，运行示例因子研究流程
3. 每天记录3个市场观察（价格异常、资金异动等），培养因子直觉

记住：**因子是冰冷的数字，但赋予它灵魂的是你对市场的理解**。就像西蒙斯所说："我们的模型其实在模拟交易员经年累月形成的条件反射"。

发布于 2025-02-02 14:38 · 广东

已赞同 218 2 条评论 796 14 分享 ··· 收起



12 项 Harness 权益全包，超量尊享 8.8 折
多模态覆盖 18+旗舰模型、Harness 权益每月赠送、多 Agent 适配生产力全面扩容 查看详情
阿里云 的广告

更多回答

Spectator 已关注

228 人赞同了该回答 〉

分享一篇使用强化学习进行因子挖掘的工作，在基准上进行测试挖掘到的因子的IC(信息系数)提升了80%，有开源代码可以直接使用。

论文 | Generating Synergistic Formulaic Alpha Collections via Reinforcement Learning

代码 | github.com/RL-MLDM/alph...

一 本文导读

在量化交易领域，将历史股票数据转化为市场趋势的指示信号是一种常见的做法，这些信号通常被称为alpha因子✦。在实践中，通常会同时使用一组公式化的alpha因子以提高建模精度，因此需要找到能够良好配合的公式化alpha因子集✦。然而，大多数传统的alpha生成器都是单独一个一个地挖掘alpha因子，忽略alpha因子组合之后产生的影响。本文提出了一种新的alpha挖掘框架，使用强化学习直接利用下游组合模型的性能来优化alpha生成器以优先挖掘一组具有协同作用的alpha因子。在A股的CSI300✦和CSI500成份股上进行测试，相对基线的表现，本文方法挖掘到的因子的IC(信息系数)提升了80%，由0.0404提升到了0.0725。

二 相关工作

发现高回报的alpha因子一直是投资者和研究人员关注的热门话题，因为alpha因子和投资收益密切相关。发现alpha因子的方法大致可以分为两类，即基于机器学习和公式化alpha因子。最近的研究主要集中在前者上。通过深度学习模型获得复杂的alpha因子，例如使用LSTM✦等序列模型，或者集成非标准数据的更复杂模型，如HIST✦和REST等。另一方面，可以用简单的公式形式表示的alpha因子是传统上由人类专家利用他们的领域知识和经验构建的，通常表达明确的经济原则。

尽管现有方法取得了显著的成功，它们仍然在不同方面存在缺点。基于机器学习的alpha因子天生复杂，有时需要比价格/成交量特征更复杂的数据。此外，虽然它们通常更具表现力，但它们往往缺乏相对较低的解释性和可解释性。因此，当这些"黑匣子"模型的性能意外地恶化时，人类专家很难相应地调整模型。因此，在某些情况下，这些算法不受欢迎，因为存在风险问题。另一方面，虽然公式化alpha因子更易于解释，但以往的研究往往集中于找到单个能够独立良好预测的alpha因子。然而，用简单规则描述股市等复杂混沌系统通常是不可能的。因此，实践中通常使用一组这些alpha因子，而不是单独使用它们。然而，用传统方法挖掘的独立alpha因子组合在一起时，最终预测性能可能不会显著提高，因为没有考虑因子之间的协同作用

问题，通过遗传挖掘alpha因子并根据某些相似性度量选择出一个子集。互信息系数（IC）通常被用作相似度“度量”。然而，将一个具有高IC的新alpha因子添加到现有alpha因子池中可能仍然会为组合结果带来非常大的性能提升，反之亦然。即使将组合模型设置为简单的线性回归器，这种现象仍然存在。因此，传统的确定一组alpha因子是否具有协同作用的方法与预期的结果不一致。

三 本文工作

本文工作提出了一个利用强化学习来改善搜索空间的探索，并使用带有约束条件的序列生成器和基于策略梯度算法✦来生成有效的公式alpha因子的框架。该框架直接基于性能优化alpha因子的生成，而不是使用传统的互信息过滤方法。

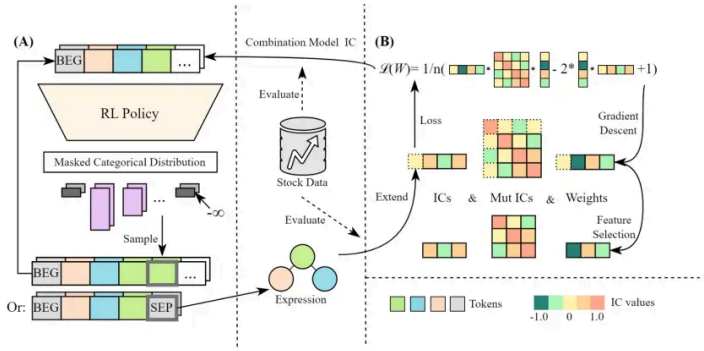


Figure 2: An overview of our alpha-mining framework. (A) An alpha generator that generates expressions, optimized via a policy gradient algorithm. (B) A combination model that maintains a weighted combination of princip... ...ceives, in the meantime, provides evaluative signals to guide the generator.

核桃量化

如上图所示，整个算法分为两部分：因子生成和因子评估，两部分交替运行。

因子生成

在股票市场中，每只股票有多维数据，每天有一个向量包含该股票在近几天的历史数据。为了研究股票的因子值，在这些数据上定义了一个与时间和具体数据无关的因子函数，作用于股票数据得到一个因子值的时间序列。这个因子函数是一个数学表达式，由算子、特征、常数、时间窗口和特殊符号组成。每个符号都与一棵表达式树等价，可以通过生成逆波兰表达式✦的方式来生成因子。

Table 1: Tokens used in our framework.

Category	Examples
Operators	*CS-Log, CS-Add, TS-Mean, , . . .*
Features	*$open, $volume, . . .*
Constants	−30, −10, −5, −2, −1, −0.5, −0.01, 0.01, 0.5, 1, 2, 5, 10, 30
Time Deltas	10*d*, 20*d*, 30*d*, 40*d*, 50*d*
Sequence Indicator	*BEG*(begin), *SEP*(end of expression)

核桃量化

上表展示了因子表达式中可以用的算子，特征，常数，时间窗口以及特殊符号。下图展示了一个因子表达式和对应的逆波兰表达式。逆波兰表示法是一种数学表达式的表示方法，它将运算符写在操作数之后，而不是写在操作数之间，从而避免了使用括号。具体来说，每个因子都可以转换为一个逆波兰表达式，其中操作数是股票数据中的特征、常数和时间窗口，操作符是算子。对于每个股票样本，在时间序列上应用逆波兰表达式，就可以得到对应的因子值序列。这种表示方法可以使因子的生成和计算更加高效和方便。在逆波兰表达式中，每个元素都必须是一个操作数或操作符，且操作数和操作符的数量必须满足特定的规则。如果表达式不符合这些规则，就是一个不合法的表达式。

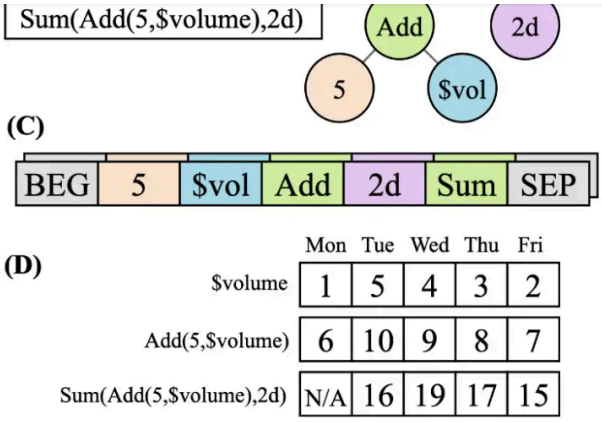


Figure 1: (A) An example of a formulaic alpha. (B) Its equivalent expression tree. (C) Its reverse Polish notation (RPN). Note that *BEG* and *SEP* are sequence indicators later mentioned in our framework. (D). Step-by-step computation of this alpha on an example time series.

核桃量化

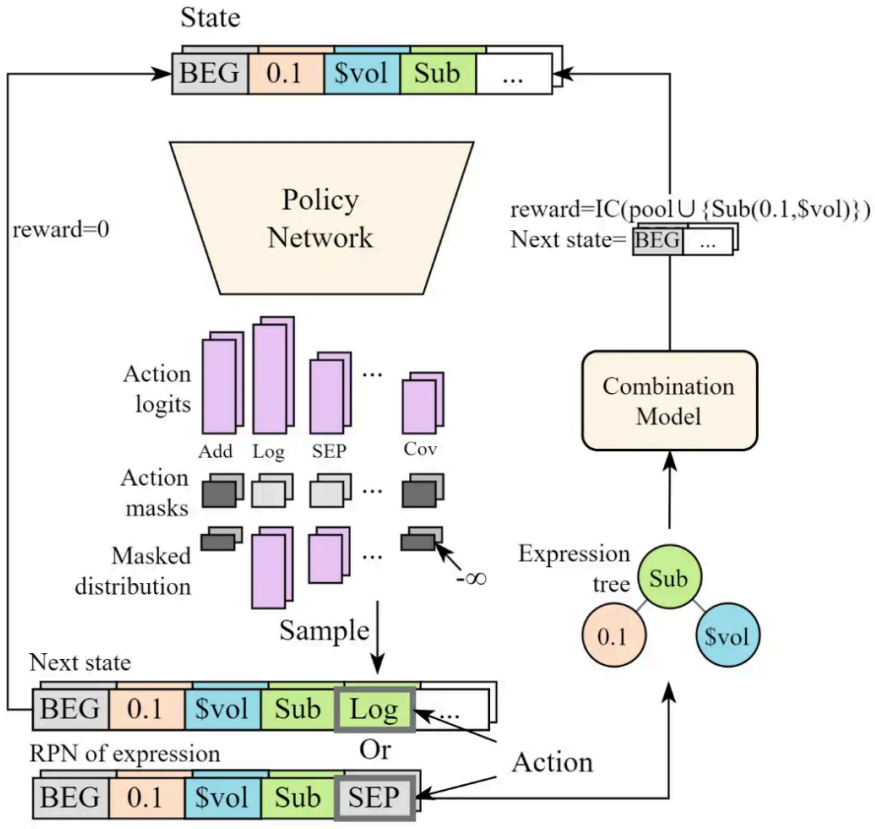


Figure 3: An illustration of our alpha generation framework.

核桃量化

本文使用强化学习来生成因子的逆波兰表达式，步骤如下：

1. 定义状态（State），包括当前已经部分生成的序列和其他相关信息（例如历史数据等）。
2. 定义行动（Action），即将要添加到序列尾部的下一个 Token。
3. 定义状态转移（Dynamic），即将行动对应的 Token 添加到状态的末尾，得到新的状态。
4. 定义奖励（Reward），即序列仅在结束时有非 0 的奖励，将序列转换为因子即为结束状态。合法因子的奖励为集成模型更新后的 IC（信息系数），非法因子的奖励为 -1。
5. 定义动作掩蔽（Action Mask），为了保证因子的形式有效性，我们能够根据状态判断一个动作是否合法，不合法的动作将被掩蔽

迭代，从而获得更高的奖励。

对应的伪代码如下：

Algorithm 2: Alpha Mining Pipeline

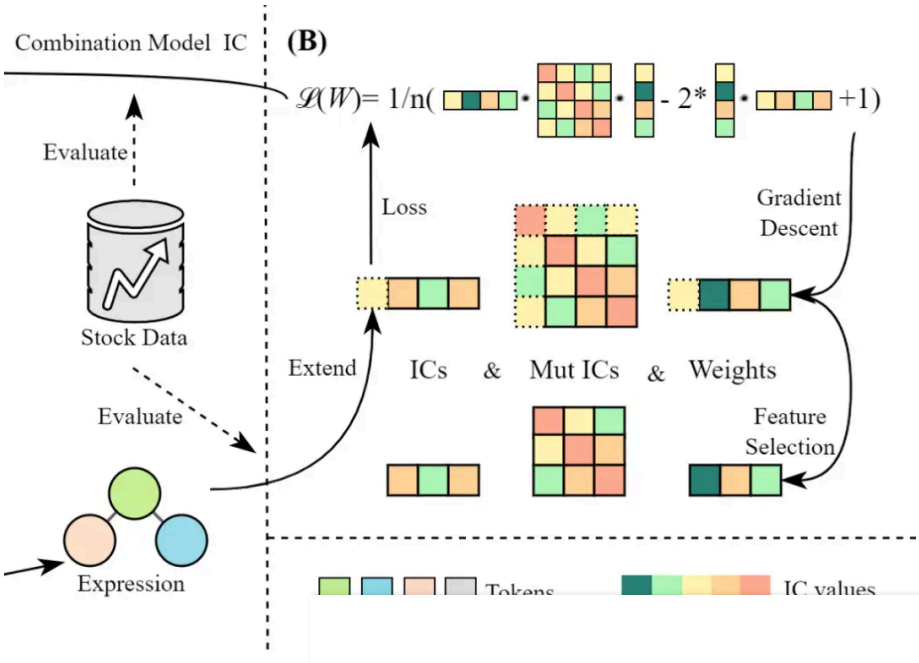
Input: Stock trend dataset $Y = \{y_t\}$

Output: Optimal alpha subset $F^* = \left\{f_1', \cdots, f_k'\right\}$, optimal weights $w^* = \left\{w_1', \cdots, w_k'\right\}$

Initialize $\mathcal{F}$ and w;
Initialize RL policy π_θ with parameters θ and replay buffer $\mathcal{D}$;
for *each iteration* **do**
  for *each environment step* **do**
    $a_t \sim \pi_\theta(a_t|s_t)$;
    $s_{t+1} \leftarrow [s_t, a_t]$;
    if a_t = SEP *or* $len(s_{t+1}) \geq threshold$ **then**
      $f \leftarrow parse(s_{t+1})$;
      Update $\mathcal{F}$, w using f and Algorithm 1;
      $IC_{new} \leftarrow \bar{\sigma}_y(\sum_{i=1}^{k} w_i f_i)$;
      $r_t \leftarrow IC_{new}$;
    else
      $r_t \leftarrow 0$;
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$;
  for *each gradient step* **do**
    Use batch $\mathcal{B} \subset \mathcal{D}$ to do gradient descent on PPO objective $\mathcal{L}^{CLIP}(\theta)$ to update θ;
return $\mathcal{F}, w$;

核桃量化

因子评估



大家都在搜 换一换
自愿放弃社保无效 615 万 热
蔡天凤失踪前曾致电多人 537 万 热
特斯拉SpaceX大跌 502 万 热
男子出狱后发现50余万仅… 473 万 热
英伟达市值9天蒸发4万亿 459 万 热
法国发生近年来最严重暴… 432 万 新
防癌神药出厂价不到1块钱 406 万
2人涉嫌操纵机器人猥亵并… 384 万 热
法考 346 万 新
三部门调查尊界刹车踏板… 309 万 热



帮助中心
服务热线：400-919-0001 帮助与客服

怎样都有一个特定的权重。

$$c(X;\mathcal{F},w)=\sum_{j=1}^{k}w_j f_j(X)=z. \tag{5}$$

核桃量化

这个集成模型的主要目的是将多个因子的信息整合在一起，以获得更准确的股票价格预测。由于不同因子的计算值可能有数量级的差别，通过对因子进行了归一化，以避免数量级差异对模型的影响。

$$\sigma(u_t,v_t)=\frac{\sum_{i=1}^{n}(u_{ti}-\bar{u}_t)(v_{ti}-\bar{v}_t)}{\sqrt{\sum_{i=1}^{n}(u_{ti}-\bar{u}_t)^2\sum_{i=1}^{n}(v_{ti}-\bar{v}_t)^2}}. \tag{1}$$

核桃量化

定义了一个损失函数来衡量模型输出和未来收益率之间的相关性，即因子值与未来收益率的Pearson相关系数✦的按日平均。这个损失函数是通过对每个股票的未来20天收益率进行统计得到的。使用梯度下降算法来更新模型中每个因子的权重，以最小化损失函数。集成模型可以是任何类型的模型，本文使用了一个线性模型，因为它易于解释和调整。下面是损失函数的详细计算公式，其中包括单个因子与未来收益率的按日平均以及因子间相互IC的按日平均。这种方法可以显著降低计算复杂度，能够在实践中更好地处理大规模数据集。

$$\mathcal{L}(w)=\frac{1}{n}\left(1-2\sum_{i=1}^{k}w_i\bar{\sigma}_y(f_i)+\sum_{i=1}^{k}\sum_{j=1}^{k}w_i w_j\bar{\sigma}(f_i(X),f_j(X))\right). \tag{7}$$

核桃量化

对应的伪代码如下：

Algorithm 1: Incremental Combination Model Optimization

Input: Alpha set $\mathcal{F}=\{f_1,\cdots,f_k\}$, weights $w=\{w_1,\cdots,w_k\}$ and a new alpha f_{new}

Output: Optimal alpha subset $\mathcal{F}^*=\left\{f_1',\cdots,f_k'\right\}$, optimal weights $w^*=\left(w_1',\cdots,w_k'\right)$

$\mathcal{F}\leftarrow\mathcal{F}\cup\{f_{\text{new}}\}, w\leftarrow w\|\text{rand}()$;
foreach $f\in\mathcal{F}$ **do**
  Obtain $\bar{\sigma}_y(f)$ from calculation or cache;
  foreach $f'\in\mathcal{F}$ **do**
    Obtain $\bar{\sigma}(f(X),f'(X))$ from calculation or cache;
for $i\leftarrow 1$ **to** *num_gradient_steps* **do**
  Calculate $\mathcal{L}(w)$ according to Equation 7;
  $w\leftarrow\text{GradientDescent}(\mathcal{L}(w))$;
$p\leftarrow\text{argmin}_i|w_i|$;
$\mathcal{F}\leftarrow\mathcal{F}\setminus\{f_p\}, w\leftarrow(w_1,\cdots,w_{p-1},w_{p+1},\cdots,w_k)$;
return $\mathcal{F}, w$;

核桃量化

四 实验结果

和CSI500指数的成分股作为股票集。作者实现了两种方法来生成alpha，分别是基于遗传编程的GP模型和基于强化学习的PPO模型。为了避免过拟合，作者构建了由两个单个alpha生成器生成的alpha集合，并使用相同的组合模型应用于这些alpha集合。为了评估模型性能，作者将其方法与在开源库Qlib[+]中实现的几个端到端机器学习模型进行比较，包括XGBoost、LightGBM和MLP[+]。

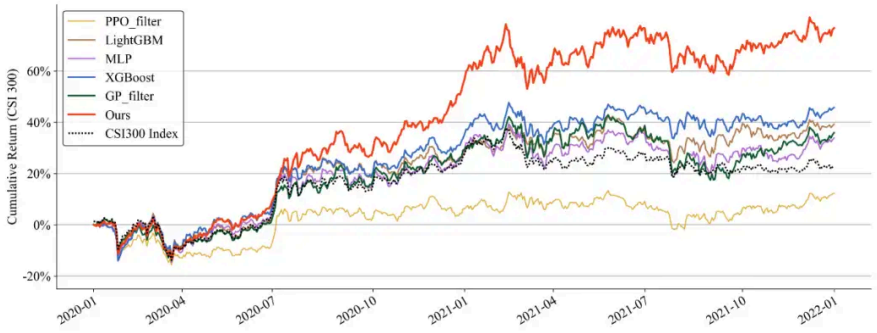


Figure 5: Backtest results on CSI 300. The lines track the net worth of simulated trading agents utilizing various alpha-mining approaches.

Table 2: Main results on CSI 300 and CSI 500. Values outside parentheses are the means, and values inside parentheses are the standard deviations across 10 runs.

Method	CSI 300		CSI 500	
	IC(↑)	Rank IC(↑)	IC(↑)	Rank IC(↑)
MLP	0.0250 (0.0068)	0.0401 (0.0081)	0.0188 (0.0018)	0.0458 (0.0045)
XGBoost	0.0404	0.0576	0.0353	0.0639
LightGBM	0.0259	0.0324	0.0332	0.0609
PPO_top*	−0.0166 (0.0028)	−0.0144 (0.0075)	0.0025 (0.0076)	0.0295 (0.0135)
GP_top*	0.0078 (0.0218)	0.0157 (0.0271)	0.0200 (0.0112)	0.0504 (0.0160)
PPO_filter*	−0.0044 (0.0107)	0.0101 (0.0107)	0.0042 (0.0042)	0.0506 (0.0052)
GP_filter*	0.0183 (0.0190)	0.0298 (0.0227)	0.0117 (0.0083)	0.0562 (0.0105)
Ours*	**0.0725** (0.0105)	**0.0806** (0.0106)	**0.0438** (0.0064)	**0.0727** (0.0112)

*Optimal combination size in {10, 20, 50, 100}

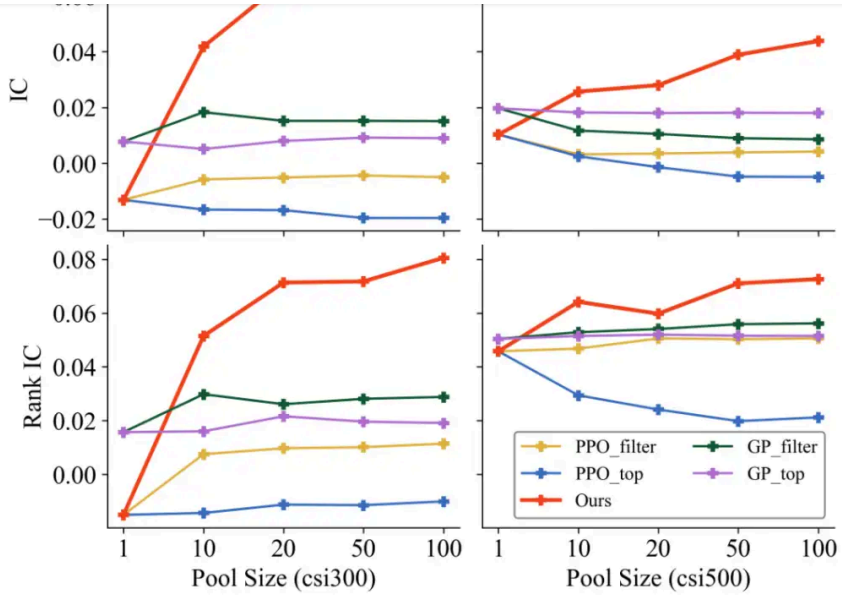


Figure 4: The results of ablation study. A pool size of 1 refers to settings that only evaluate the top-most alpha without using a combination model.

核桃量化

Table 3: An example combination of 10 alphas.

#	Alpha	Weight	IC (CSI300)
1	Var(Greater(Greater(Var(low, 50), high), open), 30)	−0.0295	0.0011
2	Max((Min(Max(close, 20) − 30, 20) + 100)/30, 20)	0.0515	0.0262
3	(Mad(high, 50) + 0.5)vwap/close	0.0343	0.0447
4	Ref(low, 50)	0.0260	0.0241
5	Min(high/close, 50) − Greater(−0.05/close, −10)	0.0437	−0.0211
6	Delta(high, 20) + high − 12	−0.0997	0.0165
7	Less(Min(2(vwap − volume), 30) + 30, 30vwap/low) − 5	0.0276	0.0025
8	Corr(Greater(Greater(vwap, volume), Greater(close, Greater(Log(Var(volume, 10)), 10))/close), close, 10)	−0.0279	−0.0338
9	|low − 30|	0.0319	0.0073
10	Max(1 − Max(Corr(low, volume · Log(10^{-4}Max(volume, 10)), 10), 10), 30)	0.0312	0.0011
	Weighted Combination		0.0511

核桃量化

五 总结展望

本文介绍了一种新的alpha挖掘框架，使用强化学习直接利用下游组合模型的性能来优化alpha生成器以挖掘具有协同作用的alpha因子。该框架通过使用序列生成器和基于策略梯度算法来生成有效的公式alpha因子，并利用强化学习来改善搜索空间的探索。

参考： [1] zhuanlan.zhihu.com/p/63...



丰川祥子
Talk is cheap.
+ 关注

参考论文链接：2023 KDD: Generating Synergistic Formulaic Alpha Collections via Reinforcement Learning

这篇文章设定了一个通过强化学习自动挖掘高组合ICIR因子的体系，定义Agent的action为不同操作符的张量化表达，state为一系列操作符的组合序列，通过RPN将表达式序列转换成最后的因子张量。对于挖掘产生的新因子，首先通过梯度下降优化该因子和现有因子库组合的最大ICIR，然后把ICIR作为Reward反馈给Agent来进行梯度下降更新PPO使用的策略网络。

本文节选其中的一小部分，旨在观察传统的符号因子表达式体系可以怎么在PyTorch上进行实现。

Expression类的定义以及子类

首先需要明确的是，不论是特征还是操作符，都是基于Expression这个父类进行实现的。具体的来说，一个表达式类的实例（Expression），都是基于一个或者两个尺寸为 $\boldsymbol{T} \times \boldsymbol{S}$ 的张量进行实现的，其中，T即为时间长度，S为股票池大小。举个例子，假如定义Close为一个Expression的实例，则调用类方法evaluate，得到的是一个张量切片，表示所有交易日所有股票的收盘价的二维张量面板

阅读全文

▲ 赞同 230 ▼ 5 条评论 885 36 分享 ···

查看全部 58 个回答